🔄 LESSON 07: LOOPS

Automating Repetition Like a Pro

Module: Control Flow Mastery

Neon City Cyber Academy

Junior Agent Training Protocol

👥 Your Team Today

Emily Chen - Handler

> "Loops are the heart of automation. Master them, and you'll never write repetitive code again."

Cole Martinez - Tech Lead

> "Think of loops as your code's autopilot. Set the conditions right, and watch it fly."

📝 Quick Review: Where We Left Off

In Lesson 06, we mastered **Nested Conditionals**:

- Security clearance systems with multiple levels
- Nested `if/elif/else` structures
- Complex decision trees

Today's Mission: Learn how to make your code **repeat automatically** using loops!

🎯 What Are Loops?

A **loop** is a programming construct that repeats a block of code multiple times.

Real-World Analogy:

- 🏃 Running laps around a track
- 🔁 Checking your inbox every 5 minutes
- 🎮 Game loops that update 60 times per second

Why Loops Matter: They eliminate repetitive code and enable automation.

🚫 The Problem Without Loops

Imagine printing numbers 1 to 100:

```
print(1)
print(2)
print(3)
# ... 97 more lines ...
print(100)
```

Problems:

- 😰 100 lines of nearly identical code
- 🐛 Hard to maintain
- ⏱️ Waste of programmer time

✅ The Solution: Loops

With a loop:

```
for i in range(1, 101):
    print(i)
```

Result: 2 lines instead of 100!

This is the power of iteration.

🔄 Two Types of Loops in Python

1. **`while` loops** - Repeat *while* a condition is true
2. **`for` loops** - Repeat *for* a specific number of times

Both are powerful, but used in different scenarios.

🌀 While Loops: The Basics

Syntax:

```
while condition:
    # code to repeat
```

How it works:

1. Check if `condition` is `True`
2. If yes → execute the code block
3. Go back to step 1
4. If no → exit the loop

🎯 Example: Countdown Timer

```
count = 5
while count > 0:
    print(f"T-minus {count}...")
    count = count - 1
print("🚀 Launch!")
```

Output:

```
T-minus 5...
T-minus 4...
T-minus 3...
T-minus 2...
T-minus 1...
🚀 Launch!
```

💬 Emily Explains: Loop Components

> "Every `while` loop has three critical parts:
>
> 1. **Initialization** - Set your starting value (`count = 5`)
> 2. **Condition** - When to keep looping (`count > 0`)
> 3. **Update** - Change the value each iteration (`count = count - 1`)
>
> Miss any of these, and you'll have problems!"

🚨 Danger: Infinite Loops

What happens if you forget the update?

```
count = 5
while count > 0:
    print("This will never stop...")
    # Forgot: count = count - 1
```

Result: The loop runs FOREVER! 💀

Your computer: 😵 (frozen)

🛡️ How to Escape Infinite Loops

In most environments:

- Press `Ctrl + C` in terminal
- Click "Stop" button in your IDE
- Wait for timeout (if running online)

Prevention: Always ensure your loop condition will eventually become `False`.

💬 Cole's Pro Tip: Debugging Loops

> "When debugging loops, add a print statement to see your loop variable:
>
> ```
> while count > 0:
>     print(f"DEBUG: count = {count}")
>     # rest of your code
> ```
>
> This helps you track if your variable is updating correctly."

🔢 For Loops: Counted Repetition

Syntax:

```
for variable in sequence:
    # code to repeat
```

Most common use: Iterating with `range()`

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

📊 Understanding range()

`range()` generates a sequence of numbers:

Code	Result
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8

Format: `range(start, stop, step)`

- `start` - First number (default: 0)
- `stop` - Stop *before* this number
- `step` - Increment (default: 1)

🎯 Example: Multiplication Table

```
number = 7
for i in range(1, 11):
    result = number * i
    print(f"{number} x {i} = {result}")
```

Output:

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

💬 Emily's Strategy: When to Use Each Loop

> "**Use `while` loops when:**
>
> - You don't know how many iterations you need
> - You're waiting for user input
> - The loop depends on a changing condition
>
> **Use `for` loops when:**
>
> - You know exactly how many times to repeat
> - You're iterating over a sequence
> - You need a counter variable"

🔄 While vs For: Side by Side

Print 1 to 5 with `while`:

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

Print 1 to 5 with `for`:

```
for i in range(1, 6):
    print(i)
```

Both produce: `1 2 3 4 5`

🎮 Challenge #1: Sum Calculator

Mission: Calculate the sum of numbers 1 to 100.

Hint: Use a loop and an accumulator variable.

```
total = 0
# Your loop here
print(f"Sum: {total}")
```

Expected Output: `Sum: 5050`

🎯 Solution: Sum Calculator

```
total = 0
for i in range(1, 101):
    total = total + i
print(f"Sum: {total}")
```

Computer Science Insight:

This is calculating **Σ(n)** where n=100.

The formula: `n * (n + 1) / 2 = 100 * 101 / 2 = 5050`

💬 Cole's Hardware Insight: How CPUs Execute Loops

> "When your CPU runs a loop:
>
> 1. It loads the loop counter into a **register** (ultra-fast memory)
> 2. Executes the loop body
> 3. Increments the counter
> 4. Checks the condition with a **compare instruction**
> 5. **Jumps** back to step 2 if condition is true
>
> This is called a **jump instruction** - the foundation of all loops in machine code!"

🎮 Challenge #2: Countdown with Input

Mission: Ask the user for a number, then countdown from that number to 1.

```
start = int(input("Enter countdown start: "))
# Your loop here
print("Blast off! 🚀")
```

Example:

```
Enter countdown start: 3
3
2
1
Blast off! 🚀
```

🎯 Solution: Countdown with Input

```
start = int(input("Enter countdown start: "))
while start > 0:
    print(start)
    start = start - 1
print("Blast off! 🚀")
```

Note: We use `while` here because the number of iterations depends on user input!

🐛 Common Errors: Off-by-One Mistakes

Problem: Loop runs one too many or one too few times.

```
# Wrong: prints 0-4 (only 5 numbers)
for i in range(5):
    print(i)

# Correct: prints 1-5 (5 numbers)
for i in range(1, 6):
    print(i)
```

Remember: `range(5)` means "0 through 4", not "1 through 5"!

🐛 Common Error: Forgetting Indentation

```
for i in range(3):
print(i)  # ❌ IndentationError
```

Fix:

```
for i in range(3):
    print(i)  # ✅ Properly indented
```

Python Rule: Code inside a loop MUST be indented!

💬 Emily's Debugging Checklist

> "When your loop isn't working:
>
> 1. ✅ Is my loop variable initialized? (`while` loops)
> 2. ✅ Is my condition correct?
> 3. ✅ Is my loop variable being updated?
> 4. ✅ Is my code properly indented?
> 5. ✅ Am I using the correct range bounds?"

🎮 Final Challenge: Password System

Mission: Create a password checker that gives the user 3 attempts.

```
password = "cyber2026"
attempts = 3

# Your loop here

if attempts == 0:
    print("🚨 Account locked!")
```

🎯 Solution: Password System

```
password = "cyber2026"
attempts = 3

while attempts > 0:
    guess = input("Enter password: ")
    if guess == password:
        print("✅ Access granted!")
        break
    else:
        attempts = attempts - 1
        print(f"❌ Wrong! {attempts} attempts left.")

if attempts == 0:
    print("🚨 Account locked!")
```

New concept: `break` (we'll cover this in Lesson 08!)

🎓 Key Takeaways

- ✅ **Loops** automate repetitive tasks
- ✅ `while` **loops** repeat while a condition is true
- ✅ `for` **loops** repeat a specific number of times
- ✅ `range()` generates number sequences
- ✅ Always ensure loops will eventually **terminate**
- ✅ Watch out for **infinite loops** and **off-by-one errors**

🚀 Next Mission: Lesson 08

Coming Up:

- Advanced loop control: `break` and `continue`
- Advanced `range()` patterns
- Loop optimization techniques
- Real-world loop applications

Agent Status: Loop Fundamentals MASTERED 🎯

📊 Mission Stats

Concept: Loops (while & for)

Difficulty: ★★★

Skills Unlocked:

- Iteration
- Automation
- Counter variables
- Loop control

Next Level: Advanced Loop Techniques